

CSCE 713 SOFTWARE SECURITY · MODULE 07 · LECTURE NOTES

Static Analysis

What it finds, and what it cannot.

MODULE OUTCOMES

- Explain the soundness and completeness trade.
- Explain taint analysis and its precision limits.
- Triage findings systematically.
- Write a custom query for a project-specific rule.
- Deploy analysis so that it is actually used.

1 The trade

RICE'S THEOREM FORCES EVERY ANALYSER TO CHOOSE WHICH WAY TO BE WRONG

CSCE 627 established that every non-trivial semantic property of programs is undecidable — so no analyser can decide, for all programs, whether a given defect is present. This is not an engineering limitation that better tools will overcome.

So each tool must choose: report everything that might be a defect and accept false positives (a sound analysis, which misses nothing), or report only what it is confident about and accept false negatives (an unsound analysis, which misses things).

And in practice nearly every usable tool chooses unsound — because a sound analyser run on real code produces far more warnings than anybody will read, which makes it unsound in effect once the team starts ignoring the output. The theoretically stronger choice is operationally weaker.

So the honest framing is: a static analyser is a heuristic defect finder with a configurable precision — not a verifier — and therefore 'the analyser passed' establishes very little about the code, which is Module 13's subject and the reason this framing matters.

	False positive	False negative
What it means	Reported, and not actually a defect.	A real defect, and not reported.
What it costs	Triage time, and the tool's credibility.	A shipped vulnerability.
The failure mode	The tool gets ignored, then disabled — which forfeits the true positives too.	False confidence, which is worse than no tool because it displaces other methods.

	False positive	False negative
Tolerable rate	Low — roughly one true positive in five findings is about the limit of what a team will sustain.	Unknown, which is precisely the problem.

The asymmetry is that you can measure the false positive rate and cannot measure the false negative rate. So the commercial and organisational pressure is always toward quieter tools, regardless of whether quieter is correct — a vendor is judged on noise and never on what was missed. **Which is CSCE 701 Module 12's measurement problem, in the tooling market**, and it is worth knowing when reading a product comparison.

2 Taint analysis

```
SOURCES      where untrusted data enters
SINKS        where it must not arrive unvalidated
SANITISERS   what makes it safe in between
```

The analyser propagates a "tainted" marking from sources through assignments and calls, and reports any tainted value that reaches a sink.

WHERE PRECISION IS LOST

```
aliasing      two names for one object; the
               analyser must guess
containers    taint one element, and most tools
               taint the whole collection
reflection    the call target is not statically
               known
callbacks     and function pointers, same problem
cross-language and any boundary the tool cannot
               follow at all
```

WHICH IS WHY findings cluster in dynamic code, and why the sanitiser list needs project-specific configuration (section 4).

Aliasing is the fundamental precision limit — precise alias analysis is itself undecidable, so every tool approximates, and the approximation is where both false positives and false negatives come from. **This is CSCE 605 Module 11's dataflow machinery applied to a different question**: the same lattices, the same fixpoint computation, the same soundness-precision trade, pointed at defects instead of at optimisations — which is worth noticing, because it means the theory transfers directly.

WHICH CLASSES STATIC ANALYSIS FINDS WELL, AND WHICH IT DOES NOT

Found well: integer defects with a short source-to-sink path (Module 03 §3's five sinks), injection with a recognisable sink (Module 04), use of a known-dangerous library function, missing error checks, and simple resource leaks.

Found poorly: use-after-free across function boundaries (Module 02 §1's locality problem), concurrency defects (Module 05 §4), and anything requiring the tool to know a project-specific invariant — which is §4's opportunity.

Not found at all: missing authorisation checks, logic errors, and design flaws — because the tool does not know what the program was supposed to do, and no amount of analysis sophistication supplies that.

So pair it with fuzzing, which has close to the opposite profile (Module 08) — **and with review, which is the only method that addresses the third category at all** (Module 11). **The three techniques are complementary rather than redundant**, which is why Project 2 requires two of them and Module 13 asks what all three together still miss.

3 Triage

- **Start with one rule, not the whole ruleset. Enable the single highest-confidence check, fix every instance of it, then add the next** — which is **the only approach that works on an existing codebase**, because enabling everything produces thousands of findings and an immediate decision to ignore them.
- **Baseline the existing findings and fail the build only on new ones**, so that **the tool constrains new code from today** rather than after a cleanup project that will never be scheduled.
- **Record a reason for every dismissal**, in the code as a suppression with a comment explaining why — **'probably a false positive' is not a triage**, and Project 2 grades it as an untriaged finding.
- **And measure the per-rule true positive rate**, then **disable the rules that never find anything real** — which is CSCE 701 Module 09 §3's rule retirement, applied to a different tool.
- **Which is the same discipline as security alert tuning — because it is the same problem**: a detector with a poor base rate, a finite human review budget, and a failure mode of being ignored. **Baseline plus fail-on-new is the deployment pattern that works**, and it avoids the cleanup project that kills most adoptions.

4 Custom queries, and deployment

THE HIGHEST-VALUE RULES ARE THE ONES ONLY YOU CAN WRITE

The built-in rules encode general knowledge, which every project shares — so they find the general defects, and **somebody has usually already run a similar tool over your dependencies**, which means the marginal yield on a mature codebase is modest.

Your project-specific invariants are in no vendor's ruleset: **'every request handler must call check_permission before touching the database'**, **'this function must never be called with a raw path'**, **'all queries go through this wrapper'**, **'no new call sites for this deprecated API'**.

And those are exactly the rules that prevent the defect you just fixed from recurring — which makes **a completed fix the right trigger for writing one**, while the mechanism is still fresh and the cost of writing the query is lowest.

So the practice is: **after fixing any defect, ask whether a query could have found it, and if so write that query into CI** — which converts an incident into a permanent guarantee and is the single highest-return use of a static analysis platform. **CodeQL and Semgrep both exist primarily for this**, and both are usable within a day.

- **Run it in CI on every change**, and **report findings on the diff rather than on the whole tree** — which makes the output proportionate to what the author just did.
- **Report into the code review**, where the author is already thinking about that specific code — **rather than into a dashboard nobody opens**, which is where most static analysis output goes to be ignored.
- **Keep the run fast enough not to be skipped**, which frequently means a quick ruleset on every change and the full analysis nightly — a slow check gets bypassed under deadline pressure, every time.

- **And own the rules.** A ruleset that nobody is responsible for tuning degrades into noise within about a year, as the codebase changes around it and the suppressions accumulate unexamined.
- **Which is CSCE 701 Module 11 §1's compliance budget — the tool spends developer attention, and that budget is finite and shared with every other process you impose. Findings in the review rather than in a dashboard is the single change that most improves whether static analysis has any effect at all.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Møller & Schwartzbach — Static Program Analysis (free PDF)	§1 and §2's theory — free, rigorous, and explicit about the precision limits.
CodeQL documentation and the query examples (free)	§4's practice — and the query language is worth learning for the project-specific rules alone.
Semgrep registry and rule-writing guide (free)	§4 with a much shallower learning curve — the right starting point for a first custom rule.
Bessey et al. — A Few Billion Lines of Code Later (free)	§3's deployment reality , from the people who commercialised it — honest about why adoption fails, and the best thing written on this.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State Rice's theorem** and explain what it forces on an analyser.
- 2 **Run a sound and an unsound analyser** on the same code and compare the output volume.
- 3 **Write a taint-flow defect** and confirm the analyser finds it.
- 4 **Add an alias in between** and see whether it still does.
- 5 **Put the tainted value in a container** and check again.
- 6 **Classify ten findings** as true positive, false positive, or undetermined, with a reason for each.
- 7 **Baseline a codebase** and configure fail-on-new.
- 8 **Write one custom query** encoding a project-specific invariant.
- 9 **Fix a defect, then write the query** that would have caught it.
- 10 **Move your findings into code review** and report whether response changed.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does Rice's theorem force, and which choice do real tools make?
- 2 Why is a sound analyser unsound in effect?
- 3 Give the two failure modes and the asymmetry between them.
- 4 Explain the source-sink-sanitiser model.
- 5 Name five places taint analysis loses precision.
- 6 Why is aliasing the fundamental limit?
- 7 Which classes are found well, poorly, and not at all?
- 8 Give four triage practices and the deployment pattern that works.
- 9 Why are custom queries higher value than the defaults?
- 10 What is the right trigger for writing one?